

JTS Topology Suite

Developer's Guide

Version 1.4, rebuilt

Base text: 17 October 2003 · rebuild: 16 August 2026

The JTS Topology Suite is a Java API for spatial data operations. This manual keeps the 2003 Developer's Guide chapter plan (getting started, relate, overlay, buffer, polygonize, line merge, custom coordinates, tips) and adds a short honest account of the SQL/MM Spatial (ISO/IEC 13249-3) curve types that are on this tree.

Package names in this tree are `org.locationtech.jts`. The 2003 copies used `com.vividsolutions.jts`. LocationTech JTS remains the upstream project; this fork is not described as if it were already upstream.

Document change control

Revision	Date	Author	Change
1.4	17 Oct 2003	Jonathan Aquino	Initial draft, created for JTS 1.4.
1.4-rebuild	16 Aug 2026	This tree	LaTeX rebuild of the 2003 guide. Curve, I/O, overlay, and Hausdorff claims match the code on this tree. ACME licence text is not the project licence.

Contents

1 Overview	3
1.1 Other resources	3
2 Getting started	3
3 Computing spatial relationships	4
4 Computing overlay operations	5
5 Computing buffers	7
5.1 Basic buffering	7
5.2 End cap styles	7
5.3 Specifying the approximation quantization	8
6 Polygonization	8
7 Merging a set of LineStrings	8
8 Using custom CoordinateSequences	9
9 Tips and techniques	9
9.1 Noding a set of LineStrings	9
9.2 Unioning many polygons efficiently	10
10 Curve types on this tree	10
11 WKT and WKB on this tree	10
12 Hausdorff on this tree	12

13 Licence**12**

1 Overview

The JTS Topology Suite is a Java API that implements a core set of spatial data operations using an explicit precision model and robust geometric algorithms. It provides a complete model for specifying 2-D linear Geometry. Many common operations in computational geometry and spatial data processing are exposed in a clear, consistent and integrated API. JTS is intended to be used in the development of applications that support the validation, cleaning, integration and querying of spatial datasets.

This document is intended for developers who would like to use JTS to accomplish their spatial data processing requirements. It describes common uses of the JTS API and gives code examples.

The 2003 note “this document is under construction” is left as history. The linear API walkthrough is still the right starting point. An opt-in curve module sits beside that model; it does not replace it.

1.1 Other resources

- OpenGIS Simple Features Specification For SQL Revision 1.1 (SFS). The reference specification for the linear spatial data model and the spatial predicates and functions implemented by JTS.
- SQL/MM Spatial (ISO/IEC 13249-3) — CircularString, CompoundCurve, CurvePolygon, and WKB codes 8–12 used by the opt-in curve module. No 13249-3 DOI is cited.
- *JTS Technical Specifications*. The design specification for the classes, methods and algorithms.
- JTS JavaDoc for packages under `org.locationtech.jts`.

2 Getting started

The most common JTS tasks involve creating and using Geometry objects. The easiest way to create a linear Geometry by hand is to use a `WKTReader` to generate one from a Well-Known Text (WKT) string. For example:

```
Geometry g1 = new WKTReader().read(
    "LINESTRING (0 0, 10 10, 20 20)");
```

A precise specification for WKT is given in the *JTS Technical Specifications*. Many examples of WKT may be found in the files in the test directory.

In a real program, it is easier to use a `GeometryFactory`, because you do not need to build up a WKT string; rather, you work with the objects directly:

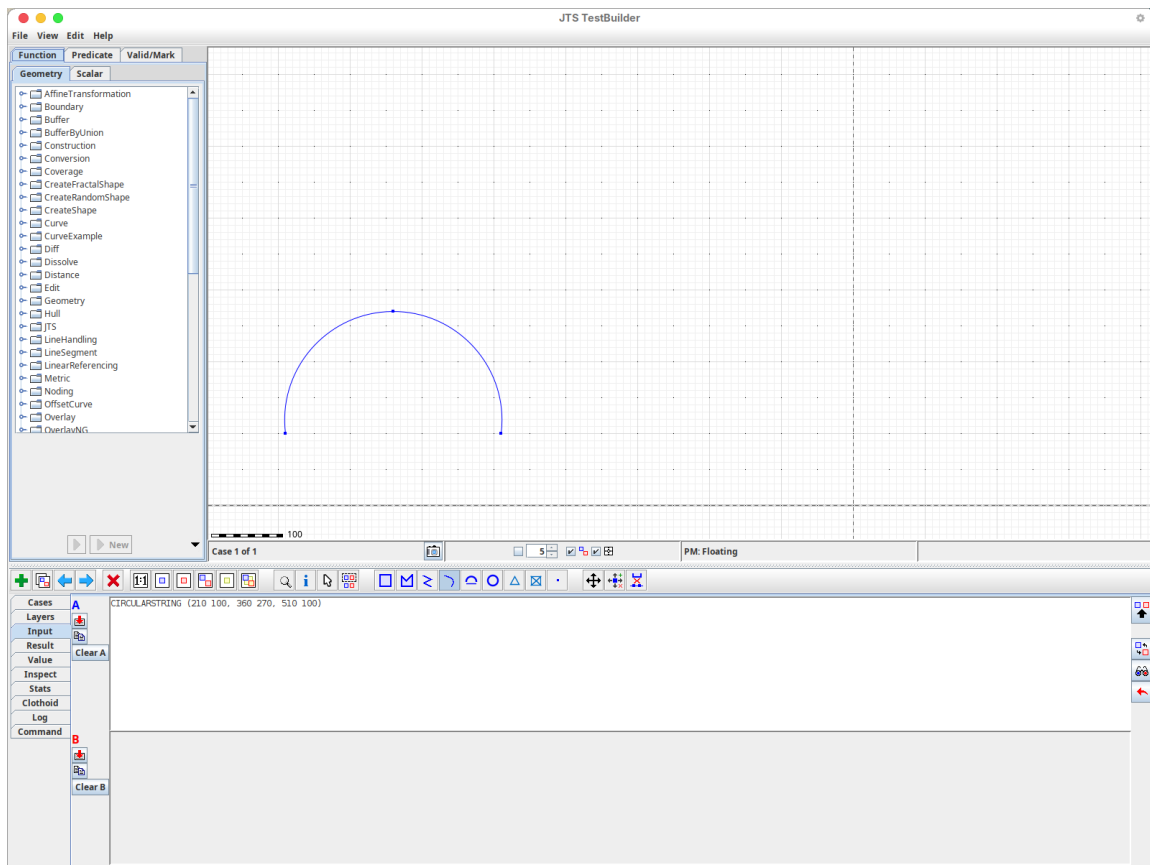
```
Coordinate[] coordinates = new Coordinate[] {
    new Coordinate(0, 0), new Coordinate(10, 10),
    new Coordinate(20, 20) };
Geometry g1 = new GeometryFactory().createLineString(coordinates);
```

Once you have a Geometry you can compute intersection, area, envelope, centroid, and buffer. See the JavaDoc for `org.locationtech.jts.geom.Geometry`.

Amendment. On this tree the concrete SQL/MM Spatial (ISO/IEC 13249-3) curve types live in the `jts-curve` module (`org.locationtech.jts.geom.curve`): `CircularString`, `CompoundCurve`, `CurvePolygon`, and the collections `MultiCurve` and `MultiSurface`. Construction is opt-in: the default `GeometryFactory` throws on `createCircularString` / `createCompoundCurve` / `createCurvePolygon` / `createMultiCurve` / `createMultiSurface`. Use `CurveGeometryFactory`. Constructors do not linearise.

A core `WKTReader` still throws `ParseException` on `CIRCULARSTRING`. Opt in:

```
CurveGeometryFactory gf = new CurveGeometryFactory();
CurveWKTReader rdr = new CurveWKTReader(gf);
Geometry arc = rdr.read("CIRCULARSTRING (0 0, 5 5, 10 0)");
// arc.getGeometryType() == "CircularString"
```



DG-1. Draw CircularString.

3 Computing spatial relationships

An important application of JTS is computing the spatial relationships between Geometries. JTS follows the Dimensionally Extended 9-Intersection Matrix (DE-9IM) model specified by the OGC. To compute the DE-9IM for two Geometries, use the `relate` method:

```

Geometry a = /* ... */;
Geometry b = /* ... */;
IntersectionMatrix m = a.relate(b);

```

Most relationships of interest can be specified as a pattern which matches a set of intersection matrices. JTS also provides a set of boolean predicates which compute common spatial relationships directly:

Method	Meaning
Equals	The Geometries are topologically equal.
Disjoint	The Geometries have no point in common.
Intersects	The Geometries have at least one point in common (the inverse of Disjoint).
Touches	The Geometries have at least one boundary point in common, but no interior points.
Crosses	The Geometries share some but not all interior points, and the dimension of the intersection is less than that of at least one of the Geometries.
Within	Geometry A lies in the interior of Geometry B.
Contains	Geometry B lies in the interior of Geometry A (the inverse of Within).
Overlaps	The Geometries share some but not all points in common, and the intersection has the same dimension as the Geometries themselves.

In some cases the precise definition of the predicates is subtle. Refer to the JTS Technical Specifications. On curve operands, public **relate** / predicates that have not taken a certified-disc path still see control-point chords unless the caller linearises explicitly via **Linearizable.toLinear(tolerance)**.

4 Computing overlay operations

The previous section discussed functions that return true or false, like **Intersects** and **Contains**. The JTS overlay operations, some of which the 2003 guide illustrated as a six-panel cartoon, are:

Method	Meaning
Buffer	The Polygon or MultiPolygon which contains all points within a specified distance of the Geometry. See Section 5.
ConvexHull	The smallest convex Polygon that contains all the points in the Geometry.
Intersection	The set of all points which lie in both A and B.
Union	The set of all points which lie in A or B.
Difference	The set of all points which lie in A but not in B.
SymDifference	The set of all points which lie in either A or B but not both.

As with the spatial relationships, these operations have precise definitions given in the JTS Technical Specifications.

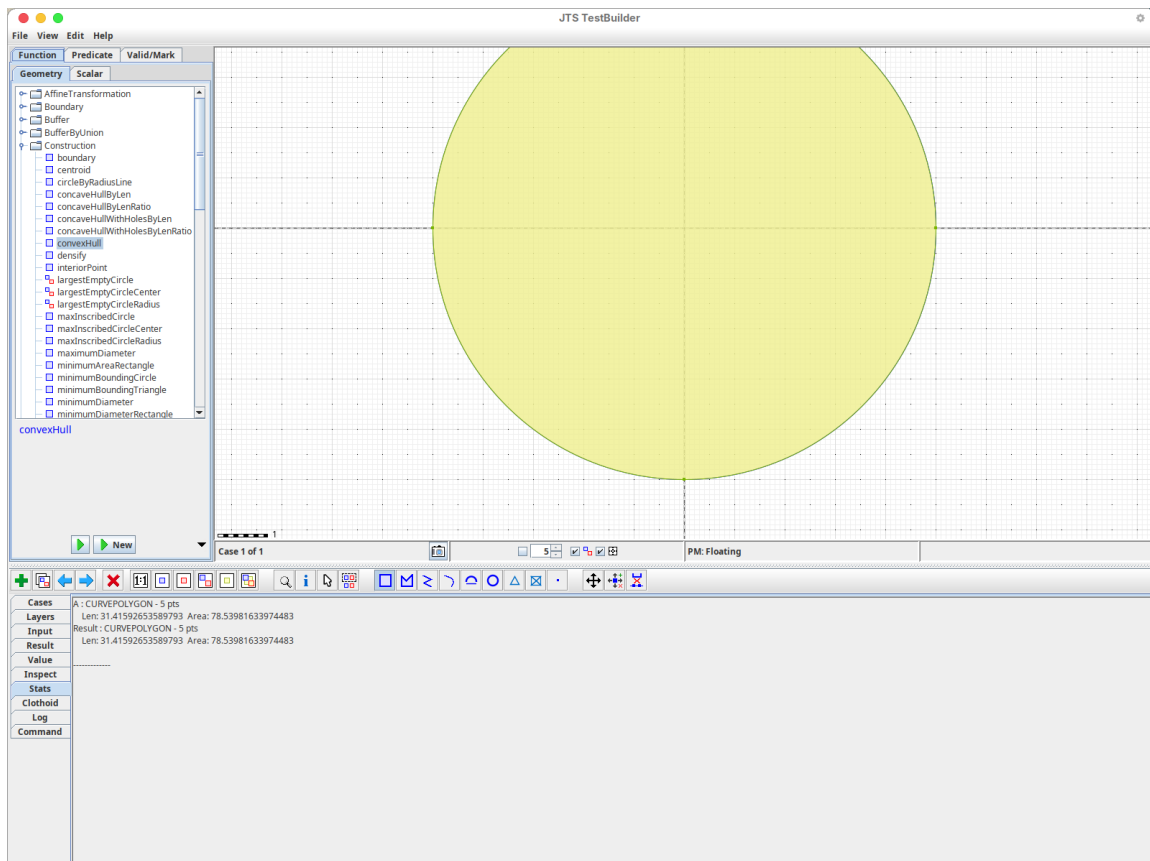
On linear Geometry the production overlay is `OverlayNG` (`org.locationtech.jts.operation.overlayng`). The 2003 noding-graph overlay is historical.

Amendment. The curve overlay type is `OverlayNGCurve`. It lives in `jts-curve` (`org.locationtech.jts.operation.overlayng.curve`). There is no public noder. Closed-form answers are limited to certified discs. Anything else densifies and delegates to core `OverlayNG`. `Geometry.intersection` / `union` / `difference` / `symDifference` on a curve type are not a general circular overlay.

Closed-form constructions on this tree are certified discs, the single-arc hull, and the compound-curve hull of circular-plus-straight members (including a stadium). Disc buffer of a full circular disc is a disc of radius $r + d$ (or empty if $r + d \leq 0$). An open arc stays on `BufferOp` chords. Anything that is not a certified closed form uses `toLinear` and the linear algorithm — a fallback, not a fail, and not a closed form.

```
import org.locationtech.jts.operation.overlayng.curve.OverlayNGCurve;
Geometry cap = OverlayNGCurve.intersection(a, b);
Geometry cup = OverlayNGCurve.union(a, b);
Geometry sub = OverlayNGCurve.difference(a, b);
Geometry xor = OverlayNGCurve.symDifference(a, b);
```

A certified disc of radius 5 has convex-hull area 25π . That hull is the disc itself, not a densified polygon.



DG-3. Disc hull area 25π . Circular-plus-straight laser hull shipped.

5 Computing buffers

In GIS, buffering is an operation used to compute the area containing all points within a given distance of a Geometry. In mathematical terms, this is computing the Minkowski sum of the Geometry with a disc of radius equal to the buffer distance. Finding positive and negative buffers is sometimes referred to as erosion and dilation. In CAD/CAM buffer curves are called offset curves.

You can use JTS to compute the buffer of a Geometry using the `Geometry.buffer` method or the `BufferOp` class. The input Geometry may be of any type (including arbitrary GeometryCollections). The result of a buffer operation is always an area type (Polygon or MultiPolygon). The result may be empty (for example, a negative buffer of a LineString).

You can compute buffers with both positive and negative buffer distances. Buffers with a positive buffer distance always contain the input Geometry. Buffers with a negative buffer distance are always contained within the input Geometry. A negative buffer of a LineString or a Point results in an empty Geometry.

Buffer distances of 0 are also supported. You can use this to perform an efficient union of multiple polygons.

5.1 Basic buffering

To compute a buffer for a given distance, call `buffer()` on the Geometry:

```
Geometry g = /* ... */;
Geometry buffer = g.buffer(100.0);
```

5.2 End cap styles

Buffer polygons can be computed with different line end-cap styles. The end-cap style determines how the linework for the buffer polygon is constructed at the ends of linestrings.

Style	Description
CAP_ROUND	The usual round end caps.
CAP_FLAT / CAP_BUTT	End caps truncated flat at the line ends (the 2003 name was CAP_BUTT).
CAP_SQUARE	End caps squared off at the buffer distance beyond the line ends.

To specify the buffer end-cap style, the `BufferOp` class in `org.locationtech.jts.operation.buffer` is used directly:

```
Geometry g = /* ... */;
BufferOp bufOp = new BufferOp(g);
```



```
bufOp.setEndCapStyle(BufferParameters.CAP_FLAT);  
Geometry buffer = bufOp.getResultGeometry(distance);
```

5.3 Specifying the approximation quantization

Since the exact buffer outline of a linear Geometry usually contains circular sections, the buffer must be approximated by the linear Geometry supported by JTS. The degree of approximation may be controlled by specifying the number of quadrant segments used to approximate a quarter-circle. Specifying a larger number of segments results in a better approximation to the actual area, but also results in a larger number of line segments in the computed polygon.

The default number of segments is 8. This gives less than a 2% maximum error in the distance of the computed curve approximation to the actual buffer curve. This error can be reduced to less than 1% by using a value of 12.

```
Geometry buffer = g.buffer(100.0, 16);
```

Amendment. A full circular disc (certified `CurvePolygon`) has a closed-form buffer: another disc of radius $r + d$, or empty if $r + d \leq 0$. An open `CircularString` is not a disc. Its buffer is `BufferOp` on chords. That is not a closed form.

6 Polygonization

Polygonization is the process of forming polygons from linework which encloses areas. Linework to be formed into polygons must be fully noded — that is, linestrings must not cross and must touch only at endpoints.

JTS provides the `Polygonizer` class to perform polygonization. The `Polygonizer` takes a set of fully noded `LineStrings` and forms all the polygons which are enclosed by the lines. Polygonization errors such as dangling lines or cut lines can be identified and reported.

```
Polygonizer polygonizer = new Polygonizer();  
polygonizer.add(lines);  
Collection polys = polygonizer.getPolygons();  
Collection dangles = polygonizer.getDangles();  
Collection cuts = polygonizer.getCutEdges();
```

If the set of lines is not correctly noded the `Polygonizer` will still operate on them, but the resulting polygonal Geometries will not be valid. The `MultiLineString` union technique can be used to node a set of `LineStrings` (see Section 9.1).

7 Merging a set of LineStrings

Sometimes a spatial operation such as union will produce chains of small `LineStrings`. The JTS `LineMerger` is a simple utility to sew these small `LineStrings` together.

The `LineMerger` assumes that the input `LineStrings` are noded (they do not cross; only their endpoints can touch). The output `LineStrings` are also noded. If `LineStrings` to be merged do not have the same direction, the direction of the resulting `LineString` will be that of the majority.

```
LineMerger lineMerger = new LineMerger();
lineMerger.add(lineStrings);
Collection merged = lineMerger.getMergedLineStrings();
```

8 Using custom CoordinateSequences

By default JTS uses arrays of `Coordinates` to represent the points and lines of `Geometries`. There are some cases in which you might want `Geometries` to store their points using some other implementation. For example, to save memory you may want to use a more compact sequence implementation, such as an array of x 's and an array of y 's. Another possibility is to use a custom coordinate class to store extra information on each coordinate, such as measures for linear referencing.

You can do this by implementing the `CoordinateSequence` and `CoordinateSequenceFactory` interfaces. You would then create a `GeometryFactory` parameterized by your `CoordinateSequenceFactory`, and use this `GeometryFactory` to create new `Geometries`. All of these new `Geometries` will use your `CoordinateSequence` implementation.

If your `CoordinateSequence` is not based on an array of the standard JTS `Coordinates` (or a subclass of `Coordinate`), it may incur a small performance penalty due to marshalling. `Curve` types use the same factory hook; `CurveGeometryFactory` can be constructed with a custom `CoordinateSequenceFactory`.

9 Tips and techniques

9.1 Noding a set of LineStrings

Many spatial operations assume that their input data is noded, meaning that `LineStrings` never cross. For example, the JTS `Polygonizer` and the JTS `LineMerger` assume that their input is noded.

The noding process splits `LineStrings` that cross into smaller `LineStrings` that meet at a point, or node. A simple trick for noding a group of `LineStrings` is to union them together; the unioning process will node the `LineStrings`.

```
Collection lineStrings = /* ... */;
Geometry noded = (LineString) lineStrings.get(0);
for (int i = 1; i < lineStrings.size(); i++) {
    noded = noded.union((LineString) lineStrings.get(i));
}
```

Unary union / `OverlayNG` is the current production path for many polygons. The 2003 “buffer of a `GeometryCollection` at distance 0” trick still works but is no longer the first recommendation.

9.2 Unioning many polygons efficiently

Calling `Polygon.union` repeatedly is one way to union several Polygons together. Prefer `UnaryUnionOp` / `OverlayNG` union on a collection. The 2003 `GeometryCollection.buffer(0)` trick remains documented as history.

10 Curve types on this tree

On this tree the concrete SQL/MM Spatial (ISO/IEC 13249-3) curve types live in the `jts-curve` module (`org.locationtech.jts.geom.curve`):

WKT keyword	Java class	Extends
CIRCULARSTRING	<code>...geom.curve.CircularString</code>	<code>LineString</code>
COMPOUNDCURVE	<code>...geom.curve.CompoundCurve</code>	<code>LineString</code>
CURVEPOLYGON	<code>...geom.curve.CurvePolygon</code>	<code>Polygon</code>
MULTICURVE	<code>...geom.curve.MultiCurve</code>	<code>MultiLineString</code>
MULTISURFACE	<code>...geom.curve.MultiSurface</code>	<code>MultiPolygon</code>

Maven artifact: `org.locationtech.jts:jts-curve`. Callers instantiate `CurveGeometryFactory` / `CurveWKTRReader` / `CurveWKBWriter` explicitly. The module does not auto-register via `ServiceLoader`.

`Linearizable.toLinear(tolerance)` is the explicit densify. Spatial methods that have not taken a certified closed form still fall through to the parent linear type or to a linearised copy. That is not hidden linearise-on-construct.

Closed-form constructions on this tree:

- Certified disc: area, envelope, disc-buffer, disc hull (radius-5 disc hull area 25π).
- Single-arc convex hull: the slice bounded by the arc and its chord, kept as a `CurvePolygon`.
- Compound-curve hull of circular-plus-straight members (H-CC), including a stadium. The exposed arc stays a `CircularString`; the result is a `CurvePolygon`, not a densified polygon. A clothoid hull stays the linear fallback (`CurveExact` null $\rightarrow$ `linearise().convexHull()`), not a laser.
- `MaximumInscribedCircle` on a certified disc and on a stadium; `LargestEmptyCircle` on legal curve obstacles. These are constructions, not a general circular noder.

The curve overlay type is `OverlayNGCurve`. There is no public noder. Closed-form overlay answers are limited to certified discs. `TestBuilder` can draw `logoLines` as curves.

11 WKT and WKB on this tree

Core `WKTRReader` still rejects `CIRCULARSTRING` and the other SQL/MM keywords. Read them with `CurveWKTRReader`. Write structured members with `CurveWKTWriter`. Core `WKTWriter` already emits the subclass keyword via `getGeometryType()` for a flat `CircularString`; composite types need the curve writer to keep member tags.

CLOTHOID is recognised only by `CurveWKTRReader`, and only as a non-leading `COMPOUNDCURVE` member. That path is extras, not a closed-form construction.

WKB type codes 8–12 are first-class in core `WKBReader` and `WKBConstants` (`CircularString=8`, `CompoundCurve=9`, `CurvePolygon=10`, `MultiCurve=11`, `MultiSurface=12`). Construction still requires a curve-capable factory; otherwise the reader throws. `CurveWKBWriter` emits codes 8–12. A bare `WKBWriter` still walks the parent `instanceof` ladder and writes a `CircularString` as `LineString` (code 2) and a `CurvePolygon` as `Polygon` (code 3). That is what a direct `WKBWriter.write` call does on this tree.

Default and export WKB paths (`JtsOp`, `TestBuilder XML export`) route through `CurveWKBWriter` so those writers do not flatten codes 8–12.

```
// Read codes 8-12 as curve types:
Geometry g = new WKBReader(new CurveGeometryFactory()).read(bytes);
Geometry g2 = new CurveWKBReader().read(bytes);

// Write codes 8-12:
byte[] wkb = new CurveWKBWriter().write(g);

// Bare writer still emits 2 / 3 for the parent types:
byte[] flat = new WKBWriter().write(g); // CircularString -> type 2
```

DG-2 WKB type 8 (`CurveWKBWriter`)

12 Hausdorff on this tree

The 2003 Developer's Guide did not document Hausdorff. It is recorded here so a reader is not left with the public-API chord behaviour as an unspoken surprise.

Public `DiscreteHausdorffDistance` on this tree (commit `0ca71b`) has a closed form for two pairs only:

1. A single-arc `CircularString` toward a single-segment `LineString`, apex $\sqrt{949}/6 - 7/6 = 3.967640600249787$ for the witness `CIRCULARSTRING (0 0, 2 3, 10 0)` vs `LINESTRING (0 0, 10 0)`.
2. Two circular discs. A `MultiSurface` unwrap of one disc is the same disc pair, not a third.

The exact path skips `densify`; `densify` is not the laser. For every other pair the public API still sees vertices / chords. Tests keep `fail()` on the general case. Fréchet remains open.



DG-4 D-HF discs

13 Licence

JTS is dual-licensed under the Eclipse Public License 2.0 and the Eclipse Distribution License 1.0. See `LICENSE_EPLv2.txt`, `LICENSE_EDLv1.txt`, and `LICENSES.md`. The 2005 TestBuilder guide's ACME licence appendix is historical and is not the project licence.

End of Developer's Guide. Remaining named holes (WKB, D-HF) are empty 4:3 slots (target 1600×1200, full TestBuilder window). 2003 Amyuni conversion cartoons are omitted; the prose they illustrated is kept.